2026-05-08-helixcode-zero-bluff-completion-design

HelixCode Zero-Bluff Completion — Design Specification

Date: 2026-05-08 **Version**: 1.0.0 **Status**: Draft (awaiting review) **Author**: agent (DeepSeek v4 Pro) **Phase**: Planning — pre-implementation

---

1. Overview

1.1 Purpose

Eliminate every remaining bluff, stub, simulation, and placeholder from the HelixCode codebase. Implement all feature gaps. Harden all tests and challenges to carry positive runtime evidence. Produce full documentation. Propagate anti-bluff governance to all submodules.

1.2 Scope

Dimension	Decision
Execution model	One comprehensive 5-phase plan, sequential
Stub scope	All documented bluffs + GAP_ANALYSIS feature gaps
Documentation	Full suite: user manual, dev guide, API ref, deployment guide, troubleshooting, challenge authoring, inline Go docs
Governance target	All 60+ submodules, including third-party
Approach	Bottom-up: governance first, then stubs, features, tests, docs

1.3 Success Criteria

1. Zero `simulated`, `placeholder`, `stub`, `TODO` in non-test production code
2. Every test carries positive runtime evidence (no absence-of-error passes)
3. Every Challenge produces sha-256 hash or equivalent verifiable evidence
4. All `t.Skip()` calls have `SKIP-OK: #<ticket>` marker
5. All 60+ submodules have anti-bluff governance
6. Full documentation suite covering 6 guides + inline Go docs
7. `go build ./...` passes on linux/amd64, darwin/arm64, windows/amd64
8. `make test-complete` runs with zero non-credential skips
9. `git status` clean across main repo and all owned-by-us submodules

---

2. Phase Architecture

```
Phase 1: GOVERNANCE ─── sets anti-bluff standard
    ↓
Phase 2: STUB ELIMINATION ─── removes all simulated code
    ↓
Phase 3: FEATURE GAPS ─── new capabilities on clean foundation
    ↓
Phase 4: TEST HARDENING ─── verifies everything works for real
    ↓
Phase 5: DOCUMENTATION ─── captures final state
```

Each phase produces a verified deliverable before the next begins. No code ships without runtime evidence.

---

3. Phase 1 — Governance Propagation

3.1 Goal

Every submodule has the anti-bluff anchor (Article XI §11.9) in its governance files.

3.2 Submodule Classification

Owned-by-us (~20 repos): Full cascade to CONSTITUTION.md / CLAUDE.md / AGENTS.md: - HelixAgent, HelixQA, HelixLLM, HelixCode (inner) - DocProcessor, LLMOrchestrator, LLMProvider, VisionEngine - LLMsVerifier, Challenges, Security, Containers

Third-party (~40+ repos): Create `.helix-governance` marker file with: - The verbatim anti-bluff anchor text - Pointer to root CONSTITUTION.md - CONST-042/043 constraints

3.3 Task Breakdown

Task	Description	Verification
P1-T01	Inventory all 60+ submodules with governance file state	Exhaustive list of paths + existing files
P1-T02	Extract exact anti-bluff anchor text from root CONSTITUTION.md	SHA-256 of anchor text for verification
P1-T03	For each owned-by-us repo: check existing files, create/update	Diff of each changed file
P1-T04	For each third-party repo: create `.helix-governance` marker	File exists with correct content
P1-T05	Update `verify-governance-cascade.sh` to cover all submodules	Script exits 0 when all pass, non-0 on gap
P1-T06	Commit all changes (deepest submodules first)	Clean push to all remotes
P1-T07		Script output

Task	Description	Verification
	Verify cascade sweep: zero submodules missing governance	

3.4 Anti-Bluff Anchor Text (Verbatim)

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"
>
> Operative rule: every PASS in this codebase MUST carry positive runtime evidence captured during execution. Metadata-only / configuration-only / absence-of-error / grep-based PASS without runtime evidence are critical defects regardless of how green the summary line looks.

---

4. Phase 2 — Stub/Bluff Elimination

4.1 Sub-phase 2A: Critical Stubs (P1)

Task	File(s)	What	TDD
P2-T01	`internal/security/security.go`	Replace `ScanFeature()` simulated result with real SonarQube/Snyk scanning via containers orchestration	Challenge expecting real scan results
P2-T02	`cmd/other_commands.go`	Wire `server`, `generate`, `test`, `worker`, `notify` to real implementations	CLI integration tests
P2-T03	`cmd/helix_config/main.go`	Audit and fix any placeholder subcommands	Config subcommand tests

4.2 Sub-phase 2B: Memory Provider Stubs (P2)

Task	File(s)	What	TDD
P2-T04	`internal/memory/providers/faiss_provider.go`	Replace 30+ simulated methods with real FAISS via cgo or subprocess	Challenge with known dataset
P2-T05		Replace simulated response	

Task	File(s)	What	TDD
	`internal/memory/providers/character_ai_provider.go`	generator with real API calls	Challenge (SKIP-OK if no creds)
P2-T06	`internal/memory/providers/anima_provider.go`	Replace simulated backup/restore with real file I/O	Challenge with checksums

4.3 Sub-phase 2C: Remaining Stubs (P2-P3)

Task	File(s)	What	TDD
P2-T07	`cmd/security_test/main.go`	Replace 12 simulated test results with real container-orchestrated scans	Challenge with real security tests
P2-T08	`internal/memory/` (Redis/Memcached)	Wire to real go-redis and Memcached clients	Integration tests against docker-compose services
P2-T09	`internal/tools/mapping/treesitter.go:266`	Implement or remove placeholder	Treesitter integration test

4.4 Sub-phase 2D: Verification

Task	Description
P2-T10	Re-verify BLUFF-004 through BLUFF-008; mark resolved in AGENTS.md
P2-T11	Remove `cmd/cli/main.go.old`; anti-bluff grep sweep; update AGENTS.md

---

5. Phase 3 — Feature Gap Implementation

5.1 Sub-phase 3A: Core Infrastructure

Task	Package	Description
P3-T01	`internal/llm/litellm/`	LiteLLM abstraction: UnifiedProvider, FormatAdapter (OpenAI/Anthropic/Google), ProviderRegistry, token counting, cost tracking
P3-T02	`internal/repomap/`	Aider-style semantic codebase mapping: tree-sitter symbols, import graphs, compressed context maps

5.2 Sub-phase 3B: Quality & Safety

Task	Package	Description
P3-T03	`internal/quality/`	Confidence scoring: compilation success, test pass rate, lint compliance, score history
P3-T04	`internal/clarification/`	Interactive clarification: ambiguity detection, structured questions, multi-turn context

5.3 Sub-phase 3C: Extensibility

Task	Package	Description
P3-T05	`internal/plugins/`	Plugin system: Plugin interface, YAML manifest, sandboxed loader, registry, hot-reload

5.4 Sub-phase 3D: Additional Providers

Task	Provider	Description
P3-T06	Cohere	Real HTTP client
P3-T07	Replicate	Real HTTP client
P3-T08	Together.ai	Real HTTP client
P3-T09	HuggingFace	Verify existing provider is real, fix if stubbed

5.5 Sub-phase 3E: Wrap-up

Task	Description
P3-T10	Update GAP_ANALYSIS.md with completion markers
P3-T11	Full build verification + anti-bluff sweep

---

6. Phase 4 — Test/Challenge Hardening

6.1 Sub-phase 4A: Audit & Classification

Task	Description
P4-T01	Full test suite audit: classify every test, identify skip/silent-pass patterns
P4-T02	Challenge harness audit: exit-code logic, expected.json assertions, runtime evidence
P4-T03	Bluff taxonomy sweep: wrapper, contract, structural, comment, skip bluffs

6.2 Sub-phase 4B: Remediation

Task	Description
P4-T04	Fix all identified bluffs with runtime evidence
P4-T05	Add `anti_bluff_verifier` challenge that scans for forbidden patterns
P4-T06	Fill challenge coverage gaps for packages without challenges

6.3 Sub-phase 4C: Infrastructure Testing

Task	Description
P4-T07	Full infrastructure test run with docker-compose.full-test.yml
P4-T08	Cross-compile verification on linux/amd64, darwin/arm64, windows/amd64

---

7. Phase 5 — Full Documentation Suite

7.1 Sub-phase 5A: New Standalone Guides

Task	Guide	Content
P5-T01	User Manual	Installation, getting started, CLI reference, TUI, configuration, providers, workflows, FAQ
P5-T02	Developer Guide	Architecture, building, conventions, adding providers/tools, contributing
P5-T03	API Reference	Generated from openapi.yaml, all endpoints, auth, WebSocket, rate limiting
P5-T04	Deployment Guide	Docker, Kubernetes, env vars, SSL, monitoring, backup, scaling
P5-T05	Troubleshooting Guide	Common errors, debug logging, health checks, connectivity, profiling
P5-T06	Challenge Authoring Guide	Structure, validation layers, evidence assertions, provider gating, anti-bluff compliance

7.2 Sub-phase 5B: Go Documentation

Task	Description
P5-T07	Go doc comments on every exported symbol in `internal/`, `cmd/`, `applications/`

7.3 Sub-phase 5C: Existing Materials Update

Task	File	Changes
P5-T08	`GAP_ANALYSIS.md`	Mark Phase 3 features COMPLETE, add evidence links
P5-T09	`HELIXCODE_FEATURE_GAP_ANALYSIS.md`	Same as T08
P5-T10	`docs/improvements/PROGRESS.md`	Document Phase 1-5 completion, close all tasks
P5-T11	`docs/CONTINUATION.md`	Set active phase, document current state
P5-T12	`AGENTS.md`	Mark all BLUFFs/ STUBs FIXED, add Phase 3 features
P5-T13	`HELIXCODE_GAP_ANALYSIS.md`	Sync with T08/T09

7.4 Sub-phase 5D: Final Verification

Task	Description
P5-T14	Documentation completeness: every feature has working challenge, no stale references
P5-T15	Final anti-bluff sweep: zero TODO/FIXME/placeholder/ simulated/stub in production code
P5-T16	Final commit + push to all 4 remotes, verify clean git status

---

8. Anti-Bluff Verification Layer

Every phase includes these mandatory verification steps before close-out:

1. **Grep sweep**: `grep -rn "simulated\|placeholder\|stub\|TODO\|for now"` — zero hits in non-test production code
2. **Skip audit**: `grep -rn "t\.Skip"` — all matches have `SKIP-OK:` marker
3. **Build**: `go build ./...` exits 0
4. **Vet**: `go vet ./...` exits 0
5. **Short tests**: `go test -short ./...` exits 0
6. **Challenge run**: all applicable challenges pass with runtime evidence
7. **Governance check**: `verify-governance-cascade.sh` exits 0

---

9. Risk Register

Risk	Likelihood	Impact	Mitigation
Third-party submodule	Medium	Low	

Risk	Likelihood	Impact	Mitigation
governance rejected by owners			`.helix-governance` marker is additive, non-invasive
FAISS native library unavailable on all platforms	High	Medium	Graceful fallback with explicit config flag
Challenges gated on credentials cannot run	High	Low	SKIP-OK markers with ticket links
60+ submodule governance too large for single session	High	Medium	Automate with scripts, batch commits
Existing tests break during stub replacement	Medium	High	TDD: failing test first, fix, verify

---

Appendix A: Anti-Bluff Anchor (Full Text)

```
Article XI §11.9 — Anti-Bluff Forensic Anchor

> Verbatim user mandate: "We had been in position that all tests do execut
> with success and all Challenges as well, but in reality the most of the
> features does not work and can't be used! This MUST NOT be the case and
> execution of tests and Challenges MUST guarantee the quality, the
> completion and full usability by end users of the product!"
>
> Operative rule: The bar for shipping is not "tests pass" but "users can
> use the feature." Every PASS in this codebase MUST carry positive runtim
> evidence captured during execution. Metadata-only / configuration-only /
> absence-of-error / grep-based PASS without runtime evidence are critical
> defects regardless of how green the summary line looks. No false-success
> results are tolerable.

### Bluff Taxonomy (each pattern observed and now forbidden)

- Wrapper bluff — assertions PASS but wrapper's exit-code logic is buggy
- Contract bluff — system advertises capability but rejects it in dispatch
- Structural bluff — file exists but doesn't contain working code
- Comment bluff — comment promises behavior code doesn't have
- Skip bluff — t.Skip("not running yet") without SKIP-OK: #<ticket> marker
```

```
The taxonomy is illustrative, not exhaustive. Every Challenge or test adde
going forward MUST pass an honest self-review against this taxonomy before
being committed.
```

Appendix B: Template Files

.helix-governance (for third-party submodules)

```
# Helix Governance Marker
#
# This file extends the HelixCode constitutional governance to this third-
# submodule. The full governance documents are at:
#
#   https://github.com/HelixDevelopment/helix_code/blob/main/CONSTITUTION.
#   https://github.com/HelixDevelopment/helix_code/blob/main/AGENTS.md
#
# Key constraints applicable to all submodules:
#
#   CONST-042: No API key, token, password, certificate, or credential may
#   committed. All secrets live in .env files (mode 0600) listed in .gitig
#
#   CONST-043: No force push, history rewrite, branch deletion of main/mas
#   or upstream-overwriting without explicit per-operation user approval.
#
#   Article XI §11.9 (Anti-Bluff): Every test PASS MUST carry positive run
#   evidence. Metadata-only / configuration-only / absence-of-error / grep
#   PASS without runtime evidence are critical defects. No false-success r
#   are tolerable. Bluff taxonomy: wrapper, contract, structural, comment,
#
# Generated: 2026-05-08 by HelixCode Zero-Bluff Completion programme
# Do not remove this file — it is verified by verify-governance-cascade.sh
```